Computer Organization & Assembly Language (CEN323)

Semester Project Proposal (Phase 1)

Contents

Project Title

Secure Message Embedding using LSB Steganography and XOR Encryption in 8086

Group Members

Name	Registration Number	GitHub Username
Syed Areeb Kareem	01-135232-095	syedareebkareem
Mujeeb Ur Rehman	01-135232-077	MUJEEBBANGASH
Marukh Jamal	01-135232-039	Mahrukh1426

Proposed Project Idea

What the project does

This project implements a secure message hiding system in 8086 Assembly Language. The program allows the user to enter a text message, encrypts it using an XOR-based cipher, and then hides the encrypted data inside an array of numbers by modifying the least significant bits (LSB). The system can later extract the hidden data from the array, decrypt it, and display the original message.

Main concept of the project

The project is based on two core concepts: steganography and encryption. Steganography is achieved by manipulating the least significant bit of each element in an array, which simulates pixel values of an image. Each character of the message is converted into binary form, and its bits are embedded into the array one by one.

Before embedding, the message is encrypted using an XOR cipher with a predefined key. During extraction, the program retrieves the bits from the array, reconstructs the encrypted characters, and applies XOR again to recover the original message. The program is structured using procedures, loops, and conditional jumps to maintain modularity and clarity.

Why is the project useful or interesting

This project demonstrates how data can be both hidden and secured using low-level programming techniques. It combines bit manipulation, encryption, and memory handling in assembly language, which strengthens understanding of how data is represented and processed at the hardware level. It is also interesting because it simulates real-world steganography techniques used in digital security, while remaining simple enough to implement in an academic environment.

Objectives

- To implement a basic steganography system using least significant bit manipulation
- To apply XOR-based encryption and decryption on user input

Features

- Input and Encryption Module: Takes user input and encrypts it using an XOR cipher
- Data Hiding Module: Embeds encrypted message bits into an array using LSB manipulation
- Extraction Module: Retrieves hidden bits and reconstructs the encrypted message
- Decryption Module: Applies XOR operation to recover the original message
- Display Module: Shows original array, modified array, and final extracted message

Technical Details

Assembly concepts to be used

The project will use loops for iterating through arrays and message characters, and conditional jumps for control flow. Bitwise operations such as AND, OR, XOR, and shift instructions (SHR) will be used for data manipulation. Registers like AX, BX, CX, DX, SI, and DI will be used for indexing and processing.

Built-in macros will not be used. The program will be written using normal assembly instructions. For modularity, separate procedures (functions) will be created for each major feature, such as encryption, data hiding, extraction, and decryption.

Interrupts/functions to be used

The project will usee DOS interrupt INT 21h for input and output operations. The following functions will be used:

1. INT 21h, AH = 01h → Read a single character from keyboard
2. INT 21h, AH = 02h → Display a single character
3. INT 21h, AH = 09h → Display a string
4. INT 21h, AH = 0Ah → Buffered string input (used to store user input in an array)
5. INT 21h, AH = 4Ch → Terminate the program

Input/output handling

User input will be taken through DOS interrupt services and stored in a buffer. Output will be displayed on the console. The program will show the original array, modified array after embedding, and the extracted message in a step-by-step manner.

Memory/data handling approach

All data will be stored in the data segment, including the simulated pixel array, message buffer, encrypted message, and extracted message. Arrays will be used to simulate image pixels. Index registers (SI and DI) will be used to traverse arrays and strings efficiently. Each character will be processed at the byte level and converted into bits for embedding and extraction.

Complexity Justification

This project is suitable for a three member team because it contains two main logical components: encryption and steganography. One member can implement the XOR encryption and decryption module, while the other focuses on the data hiding and extraction logic. Both parts require careful bit manipulation and coordination during integration. Final testing, debugging, and user interface handling will be done by the third member.

Timeline

Date / Week	Planned Tasks
May 4 - May 12	Divide the project into modules and implement them separately. One member will work on XOR encryption and decryption, while the other member will implement the steganography (data hiding and extraction). Yet to decide about what excat person will do.
May 12 - May 14	Integrate all modules, perform testing and debugging, verify correct encryption/decryption and embedding/extraction, and prepare the final submission.

References

GitHub Repository Link: https://github.com/syedareebkareem/Coal-Project

Overleaf Project Link: https://www.overleaf.com/read/sbfwfpnzyskn#63c1c6